

</> UD05-3.Spring Security - OAuth2

```
public class UD04 {  
    public static void main (String[] args) {  
        System.out.println (
```

1. Introducción.....	2
2. Autenticación mediante OAuth2 de Github.....	3
3. Configurar el proyecto para OAuth2 de Github.....	7
4. Modificación SecurityConfig.....	9
5. Modificación de las vistas.....	10
6. Security Handler.....	11

```
    );  
};  
};
```

1. Introducción

La autenticación mediante **OAuth2** es un estándar utilizado para delegar la autenticación de usuarios a proveedores externos como **Google, GitHub, Facebook,...** Esto simplifica el inicio de sesión para los usuarios, al permitirles usar sus cuentas y también mejora la seguridad de las aplicaciones al delegar la gestión de credenciales a proveedores confiables.

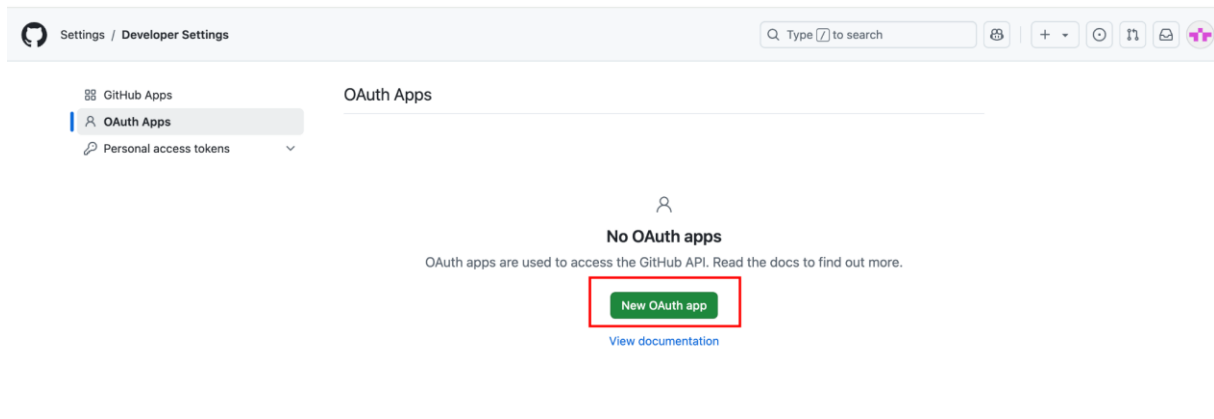
Spring Boot y **Spring Security** proporcionan una integración para implementar autenticación OAuth2 de manera sencilla. Usando el módulo `spring-boot-starter-oauth2-client`, podemos integrar proveedores OAuth2

La autenticación OAuth2 en una aplicación Spring Boot usando **un servicio externo (Google, Facebook, Github,...)** como proveedor conlleva los siguientes pasos:

- Configurar la integración con el servicio desde su consola de desarrolladores.
- Definir propiedades en `application.properties` para establecer el cliente OAuth2.
- Personalizar la aplicación con un inicio de sesión de un servicio de terceros.
- Acceder a la información del usuario autenticado.

2. Autenticación mediante OAuth2 de Github

Paso 1: Entramos en el siguiente enlace de github: [GitHub Developer Settings](#) y pulsamos el botón **New OAuth app**



Paso 2: Rellenamos los datos del formulario:

Register a new OAuth app

Application name *

dwese-ticket-logger-webapp

Something users will recognize and trust.

Homepage URL *

http://localhost:8080

The full URL to your application homepage.

Application description

Application description is optional

This is displayed to all users of your application.

Authorization callback URL *

http://localhost:8080/login/oauth2/code/github

Your application's callback URL. Read our [OAuth documentation](#) for more information.

☐ **Enable Device Flow**

Allow this OAuth App to authorize users via the Device Flow.

Read the [Device Flow documentation](#) for more information.

Register application

[Cancel](#)

Paso 3: Copia el Client ID

General

Optional features

Advanced

dwese-ticket-logger-webapp

antonio-gabriel-gonzalez-casado

 owns this application.

Transfer ownership

You can list your application in the [GitHub Marketplace](#) so that other users can discover it.

List this application in the Marketplace

0 users

Revoke all user tokens

Client ID

Client secrets

Generate a new client secret

You need a client secret to authenticate as the application to the API.

Application logo

Drag & drop

Upload new logo

You can also drag and drop a picture from your computer.

Paso 4: Genera Client Secret y copialo:

Client secrets

Generate a new client secret

Make sure to copy your new client secret now. You won't be able to see it again.



Client secret

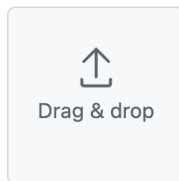
Added now by **antonio-gabriel-gonzalez-casado**
Never used



Delete

You cannot delete the only client secret. Generate a new client secret first.

Application logo



Upload new logo

You can also drag and drop a picture from your computer.

Application name *

3. Configurar el proyecto para OAuth2 de Github

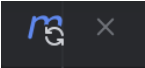
Paso 1: Añadir la dependencia de Spring para el OAuth2 en el **pom.xml** (debajo de spring-boot-starter-security):

```
<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-oauth2-client</artifactId>

</dependency>
```

Después de añadirla pulsar el botón  y hacer un **mvnw clean install**

Paso 2: Crea las siguientes variables de entorno en el archivo **.env**:

```
GITHUB_CLIENT_ID=TU_CLIENT_ID
GITHUB_CLIENT_SECRET=TU_CLIENT_SECRET
```

Sustituyendo **TU_CLIENT_ID** y **TU_CLIENT_SECRET** por los valores generados en el paso anterior.

Paso 3: Configurar las propiedades necesarias para el login OAuth2 mediante Github en nuestro **application.properties**.

```
spring.security.oauth2.client.registration.github.client-
id=${GITHUB_CLIENT_ID}

spring.security.oauth2.client.registration.github.client-
secret=${GITHUB_CLIENT_SECRET}

spring.security.oauth2.client.registration.github.scope=read:user

spring.security.oauth2.client.provider.github.authorization-
uri=https://github.com/login/oauth/authorize

spring.security.oauth2.client.provider.github.token-
uri=https://github.com/login/oauth/access_token
```

```
spring.security.oauth2.client.provider.github.user-info-  
uri=https://api.github.com/user  
  
# Este es el campo que se recuperará para mostrar en las pantallas, en este  
caso login significa el nombre de usuario  
  
# con el que se hace login  
  
spring.security.oauth2.client.provider.github.user-name-attribute=login
```

La descripción de los campos es la siguiente:

client-id y **client-secret**: Proporcionados por GitHub.

scope: Permisos solicitados. En este caso, **read:user** para acceder a la información básica del usuario.

authorization-uri: URL de autorización de GitHub.

token-uri: URL para obtener el token de acceso.

user-info-uri: URL para obtener la información del usuario autenticado.

user-name-attribute: El atributo principal que identifica al usuario (**id** en GitHub).

Los valores de estas variables están en la documentación de cada servicio con el que nos integremos.

En el caso de github aquí esta es la documentación:

<https://docs.github.com/en/apps/creating-github-apps/writing-code-for-a-github-app/building-a-login-with-github-button-with-a-github-app>

En el caso de google aquí está la documentación:

<https://developers.google.com/identity/protocols/oauth2/web-server?hl=es-419>

4. Modificación SecurityConfig

En la clase de configuración de Seguridad de Spring que hemos creado en las guías anteriores (**SecurityConfig.java**) le vamos a añadir la configuración que se necesita para el OAuth2. Esto se hará añadiendo en el método **securityFilterChain** las siguientes directivas(debajo del bloque `.formLogin(form...` y antes del `.sessionManagement(session...)`):

```
.oauth2Login(oauth2 -> {  
    logger.debug("Configurando login con OAuth2");  
    oauth2  
        .loginPage("/login")           // Reutiliza la página de inicio de  
sesión personalizada  
        .defaultSuccessUrl("/", true) // Redirige al inicio después del  
login exitoso con OAuth2  
        .permitAll();  
})
```

5. Modificación de las vistas

Paso 1: Agrega la librería de iconos de bootstrap en el archivo **head** del html debajo de la librería de bootstrap.

```
<!-- Bootstrap Icons -->

<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons/font/bootstrap-icons.css">
```

Paso 2: Agrega el código html del botón para hacer login con github en el **login.html** (debajo del div con el formulario de login actual):

```
<div class="row justify-content-center mt-3">

  <div class="col-md-6">

    <!-- Botón de inicio de sesión con GitHub -->

    <a href="/oauth2/authorization/github" class="btn btn-dark btn-lg w-100">

      <i class="bi bi-github"></i> Iniciar sesión con GitHub

    </a>

  </div>

</div>
```

Prueba que el login con github funciona correctamente:



The screenshot shows a web application interface. At the top, there is a dark navigation bar with the text "TL - NOMBRE ALUMNO" and a list of links: "Inicio", "Comunidades Autónomas", "Provincias", "Supermercados", "Ubicaciones", "Categorías", and "Tickets". On the right side of the navigation bar, there is a language selector "Español" and a "Login" link. Below the navigation bar, the main heading "Iniciar sesión" is displayed. Underneath the heading, there is a login form with two input fields: "Usuario" and "Contraseña". Below these fields is a blue button labeled "Entrar". At the bottom of the form, there is a dark button labeled "Iniciar sesión con GitHub", which is highlighted with a red rectangular box.

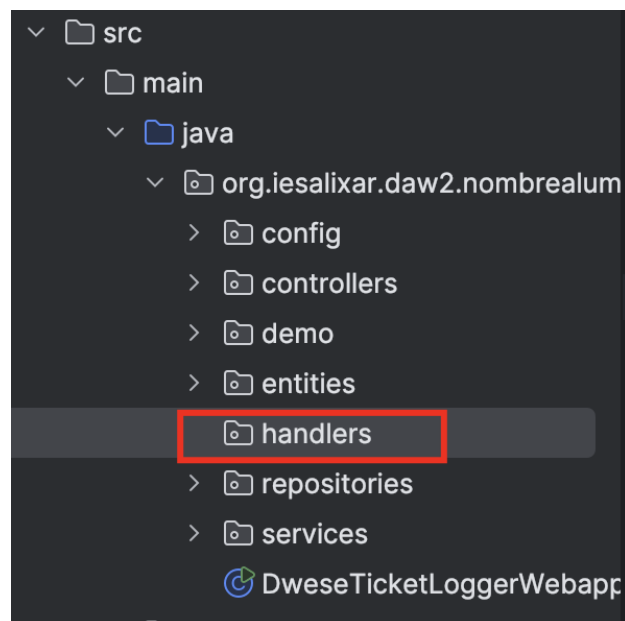
6. Security Handler

Hasta el paso anterior cualquier usuario con cuenta en github puede hacer login en nuestra aplicación, pero lo que realmente nosotros queremos es que hagan login solo los usuarios que se encuentren en nuestra tabla **users**.

Para ello vamos a definir un **Handler** que es una clase (componente) con una anotación **@Component** que se encarga de manejar eventos o acciones específicas dentro de un flujo de Spring. En el caso de Spring Security, los handlers se utilizan para personalizar el comportamiento durante ciertos eventos de seguridad, como el éxito o el fallo en la autenticación, el cierre de sesión, o los accesos denegados.

Para asegurarnos de que un usuario debe estar previamente registrado en nuestra aplicación antes de poder iniciar sesión con OAuth2, podemos implementar una lógica personalizada en un **OAuth2 Authentication Success Handler**. Este handler verifica si el usuario autenticado con un proveedor externo (como GitHub) ya está registrado en nuestra base de datos. Si no lo está, podemos rechazar el acceso o redirigirlo a una página de registro.

Paso 1: Creamos el paquete handlers



Paso 2: En primer lugar, vamos a añadir al paquete **handlers** la clase **CustomOAuth2FailureHandler.java** que contiene los pasos que se requieren para gestionar el fallo al hacer login por usuario no encontrado en nuestra base de datos.

```
package org.iesalixar.daw2.nombrealumno.dwese_ticket_logger_webapp.handlers;

import org.slf4j.Logger;

import org.slf4j.LoggerFactory;

import org.springframework.security.core.context.SecurityContextHolder;

import org.springframework.security.web.authentication.AuthenticationFailureHandler;

import org.springframework.stereotype.Component;

import jakarta.servlet.ServletException;

import jakarta.servlet.http.HttpServletRequest;

import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;

/**
 * Handler personalizado para manejar fallos en la autenticación con OAuth2.
 *
 * Este handler se encarga de limpiar el contexto de seguridad, invalidar la
 * sesión
 *
 * actual y redirigir al usuario a la página de inicio de sesión, mostrando un
 * mensaje
 *
 * de error específico en caso de que la autenticación falle.
 */

@Component

public class CustomOAuth2FailureHandler implements
AuthenticationFailureHandler {
```

```

        private static final Logger logger =
LoggerFactory.getLogger(CustomOAuth2FailureHandler.class);

    /**
     * Maneja los fallos en la autenticación con OAuth2.
     *
     * Este método se ejecuta automáticamente cuando ocurre un fallo de
autenticación.
     *
     * Realiza las siguientes acciones:
     *
     * - Limpia el contexto de seguridad.
     *
     * - Invalida la sesión actual.
     *
     * - Agrega un mensaje de error a la sesión.
     *
     * - Redirige al usuario a la página de inicio de sesión.
     *
     * @param request El objeto {@link HttpServletRequest} que contiene la
solicitud HTTP.
     *
     * @param response El objeto {@link HttpServletResponse} que contiene la
respuesta HTTP.
     *
     * @param exception La excepción de autenticación que indica el motivo del
fallo.
     *
     * @throws IOException Si ocurre un error de E/S durante la redirección.
     *
     * @throws ServletException Si ocurre un error relacionado con el manejo de
la solicitud.
     */
    @Override
    public void onAuthenticationFailure(HttpServletRequest request,
HttpServletResponse response,
org.springframework.security.core.AuthenticationException exception)
        throws IOException, ServletException {

```

```
logger.warn("Falló la autenticación: {}", exception.getMessage());

// Limpiar el contexto de seguridad
SecurityContextHolder.clearContext();

// Invalidar la sesión actual
request.getSession().invalidate();

// Agregar el mensaje de error como un atributo de sesión
request.getSession().setAttribute("errorMessage", "El usuario no está
registrado en esta aplicación");

// Redirigir al login con el parámetro de error
response.sendRedirect("/login");
}
}
```

En esta clase se invalida la sesión iniciada con github y luego se crea el mensaje de error para mostrar en el formulario de login.

Paso 3: Para poder mostrar el error vamos a tener que modificar nuestro **LoginController.java** añadiendo el parametro **HttpServletRequest request** que es el que recibe el mensaje de error que el handler ha guardado en la sesión y el parámetro **Model model** que es el que llevaría a la vista el mensaje de error:

```
/**
 * Maneja las solicitudes GET a la página de inicio de sesión.
 * Recupera mensajes de error de la sesión, si existen, y los pasa al modelo
 * para ser mostrados en la vista de login.
 * @param request El objeto {@link HttpServletRequest} que contiene la solicitud
 * HTTP.
 * @param model El objeto {@link Model} que se utiliza para pasar atributos a
 * la vista.
 * @return El nombre de la plantilla de Thymeleaf que renderiza la página de
 * login.
 */

@GetMapping("/login")
public String login(HttpServletRequest request, Model model) {
    // Recuperar el mensaje de error desde la sesión

    String errorMessage = (String)
request.getSession().getAttribute("errorMessage");

    // Si existe un mensaje de error, añadirlo al modelo y limpiar la sesión

    if (errorMessage != null) {
        model.addAttribute("errorMessage", errorMessage);

        request.getSession().removeAttribute("errorMessage"); // Limpiar el
mensaje después de mostrarlo
    }

    return "login"; // Redirige a una plantilla personalizada de login
}
```

Paso 4: Agregar en el **login.html** debajo del **h1** el código para mostrar mensajes de error que venimos usando en todas las vistas del proyecto:

```
<div th:if="{errorMessage}" class="alert alert-danger">

    <p th:text="{errorMessage}"></p>

</div>
```

Paso 5: Añadimos la clase **CustomOAuth2SuccessHandler.java**. Dentro de esta clase se está sobrescribiendo el método **onAuthenticationSuccess** que es el que se encarga de una vez se haya realizado una autenticación correcta contra github comprobar que el usuario existe en la base de datos. Esto se hace llamando al método del repositorio **existByUsername**.

```
package org.iesalixar.daw2.nombrealumno.dwese_ticket_logger_webapp.handlers;

import
org.iesalixar.daw2.nombrealumno.dwese_ticket_logger_webapp.repositories.UserR
epository;

import
org.iesalixar.daw2.nombrealumno.dwese_ticket_logger_webapp.services.CustomUse
rDetailsService;

import org.springframework.beans.factory.annotation.Autowired;

import
org.springframework.security.authentication.UsernamePasswordAuthenticationTok
en;

import org.springframework.security.core.Authentication;

import org.springframework.security.core.context.SecurityContextHolder;

import org.springframework.security.core.userdetails.UserDetails;

import org.springframework.security.oauth2.core.OAuth2AuthenticationException;

import org.springframework.security.oauth2.core.user.OAuth2User;
```



```

import
org.springframework.security.web.authentication.AuthenticationSuccessHandler;

import org.springframework.stereotype.Component;

import jakarta.servlet.ServletException;

import jakarta.servlet.http.HttpServletRequest;

import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;

/**
 * Handler personalizado para manejar eventos de éxito en la autenticación con
 OAuth2.
 *
 * Este handler verifica si el usuario autenticado con un proveedor externo (por
 ejemplo, GitHub)
 *
 * ya está registrado en la base de datos de la aplicación. Si no está registrado,
 se redirige
 *
 * al usuario a una página de registro. En caso contrario, el usuario es
 redirigido a la página principal.
 */

@Component

public class CustomOAuth2SuccessHandler implements
AuthenticationSuccessHandler {

    @Autowired

    private UserRepository userRepository;

    @Autowired

    private CustomUserDetailsService customUserDetailsService;

```

```

/**
 * Maneja el evento de autenticación exitosa con OAuth2.
 *
 * Verifica si el usuario autenticado ya existe en la base de datos de la
aplicación.
 *
 * Si no existe, se redirige a una página de registro. Si existe, se redirige
al inicio.
 *
 * @param request      Objeto {@link HttpServletRequest} que contiene la
solicitud HTTP.
 *
 * @param response      Objeto {@link HttpServletResponse} que contiene la
respuesta HTTP.
 *
 * @param authentication Objeto {@link Authentication} que representa al
usuario autenticado.
 *
 * @throws IOException      Si ocurre un error en la redirección.
 *
 * @throws ServletException Si ocurre un error en el manejo de la solicitud.
 */

@Override
public void onAuthenticationSuccess(HttpServletRequest request,
HttpServletResponse response,
Authentication authentication) throws
IOException, ServletException {
    // Obtener el objeto OAuth2User del usuario autenticado
    OAuth2User oAuth2User = (OAuth2User) authentication.getPrincipal();

    // Obtener el nombre de usuario (por ejemplo, el login de GitHub)
    String username = oAuth2User.getAttribute("login");

    // Verificar si el usuario está registrado en la base de datos
    if (!userRepository.existsByUsername(username)) {

```

```

        // Lanzar una excepción estándar de Spring Security. Esto hará que
        se gestione por el handler CustomOAuth2FailureHandler

        throw new OAuth2AuthenticationException("El usuario " + username +
" no está registrado en el sistema.");

    }

    // Cargar los detalles del usuario desde la base de datos

    UserDetails userDetails =
customUserDetailsService.loadUserByUsername(username);

    // Crear un nuevo objeto Authentication con los detalles del usuario
    cargados

    // userDetails.getAuthorities() contiene los detalles de los roles y
    permisos que se han establecido en el
    customUserDetailsService.loadUserByUsername

    UsernamePasswordAuthenticationToken authenticationToken = new
UsernamePasswordAuthenticationToken(

        userDetails, null, userDetails.getAuthorities()

    );

    // Establecer el nuevo objeto Authentication en el SecurityContextHolder

    SecurityContextHolder.getContext().setAuthentication(authenticationToken);

    // Si el usuario está registrado, redirigir a la página principal

    response.sendRedirect("/");

}

}

```

Paso 6: En la clase anterior se usan métodos del user repository que se tendrán que crear mediante la convención de Spring Data (esto no está incluido en esta guía).

Paso 7: Modificar el **SecurityConfig.java** de nuevo para cambiar el handler genérico por el nuestro personalizado. Esto se hace sustituyendo las líneas:

```
.defaultSuccessUrl("/", true) // Redirige al inicio después del login exitoso
con OAuth2

.permitAll();
```

Por estas otras que son las que invocan a los handlers que hemos creado:

```
.successHandler(customOAuth2SuccessHandler) // Usa el Success Handler
personalizado

.failureHandler(customOAuth2FailureHandler); // Handler para fallo en
autenticación
```

Donde **customOAuth2SuccessHandler** y **customOAuth2FailureHandler** los tenemos que inyectar como atributos de la clase **SecurityConfig**:

```
@Autowired
private CustomOAuth2SuccessHandler customOAuth2SuccessHandler;

@Autowired
private CustomOAuth2FailureHandler customOAuth2FailureHandler;
```

Una vez hecho todo esto prueba a hacer login con tu cuenta de github, te debería mostrar el mensaje de que no existe la cuenta de usuario:

TL - NOMBRE ALUMNO

[Inicio](#) [Comunidades Autónomas](#) [Provincias](#) [Supermercados](#) [Ubicaciones](#) [Categorías](#) [Tickets](#)

Español

Login

Iniciar sesión

El usuario no está registrado

Usuario

Usuario

Contraseña

Contraseña

Entrar

Iniciar sesión con GitHub

IES ALIXAR. DAW2. Desarrollo Web Entorno Servidor. Nombre Alumno

Después agrega tu usuario de github en la base de datos en la tabla **users** y añádele también algún rol en la tabla **users_roles** y vuelve a probar a hacer login:

TL - NOMBRE ALUMNO

[Inicio](#) [Comunidades Autónomas](#) [Provincias](#) [Supermercados](#) [Ubicaciones](#) [Categorías](#) [Tickets](#)

Bienvenido,
casado

antonio-gabriel-gonzalez-

Español

Logout

Ticket Logger

NOMBRE ALUMNO - Aplicación Web para el registro de tickets en compras de supermercado

IES ALIXAR. DAW2. Desarrollo Web Entorno Servidor. Nombre Alumno